2026 년 1 학기 네트워크프로그래밍 중간시험

☞ 주의 사항

- 아래의 조건(TCP 소켓, 구조체)을 따르지 않고 구현한 경우, 0 점 처리함
- 기능 구현과 상관없이 프로그램 동작 중 예외가 발생되는 경우, 각 항목별 2 점 감점함
- 각 기능별 부분 점수는 없음 (기능에 대한 코드만 있고 동작이 되지 않으면 점수 없음)
- 문자열 출력 부분에서 쓰레기 값이 출력되면 4 점씩 감점
- 문제에서 제시한 조건과 맞지 않게 구현한 경우, 항목당 2 점씩 감점함
- 총 4 개의 cmd type 을 조건문을 이용해서 확인하지 않는 경우, 항목당 2 점씩 감점함
- 컴파일 에러 발생시 0 점 처리함
- 각 소스 파일에 학번, 이름(영문 가능) 주석이 없는 경우, 파일당 1 점씩 감점함

1. TCP 통신을 이용한 패킷 재조립(Packet Reassembling) 프로그램 구현 (30 점)

제출파일: server.c client.c

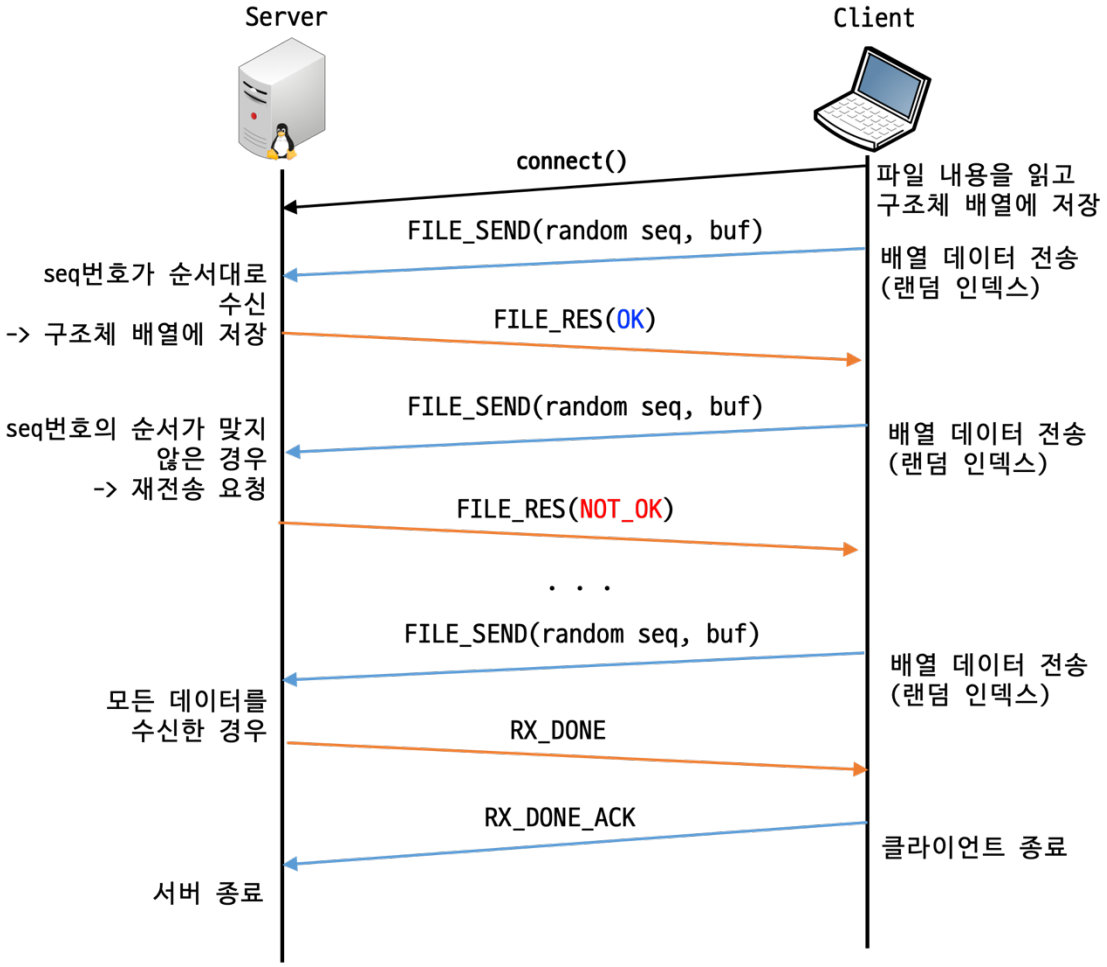
클라이언트는 텍스트 파일(message.txt)을 읽고 구조체 배열(DATA data[MAX_LINE])에 seq 번호 및 파일 내용을 저장한다. 클라이언트는 랜덤 인덱스를 생성하여 구조체 배열 내 임의의 데이터를 서버로 전송하며, 서버는 수신된 seq 번호가 기대한 순서가 아닐 경우 구조체 배열(DATA file_data[MAX_LINE])에 저장하지 않고 NOT_OK 값을 전달한다. 서버는 정상적으로 수신된 seq 패킷의 경우, 구조체 배열에 해당 데이터를 저장하도록 프로그램을 구현하시오.

■ 데이터 전송 구조체

```
#define BUF_SIZE 100
#define MAX_LINE 10     // 파일의 라인 길이

#define FILE_SEND 1   // 랜덤 데이터 전송: Client -> Server
#define FILE_RES 2    // FILE_SEND 의 응답: Server -> Client
#define RX_DONE 3     // 모든 데이터 수신 완료: Server -> Client
#define RX_DONE_ACK 4 // 수신 완료에 대한 ACK: Client -> Server

// RES_PACKET 의 result 값
#define NOT_OK 0  // 순서가 맞지 않는 SEQ 를 가지는 패킷에 대한 응답
#define OK 1      // 정상 전송된 SEQ 패킷 응답

typedef struct
{
    int seq;
    char buf[BUF_SIZE];
} DATA;

// Client -> Server 전송
typedef struct
{
```

```
    int cmd; // TX_DATA, RX_DONE_ACK
    int seq; // 랜덤하게 선택한 배열 인덱스에 저장된 seq 번호
    char buf[BUF_SIZE];
} REQ_PACKET;

// Server -> Client 전송
typedef struct
{
    int cmd;    // RES_DATA, RX_DONE 전송
    int seq;    // 수신한 seq 번호
    int result; // 0: 재전송, 1: 수신 Ok
} RES_PACKET;
. . .
DATA file_data[MAX_LINE]; // 파일 데이터 저장용 구조체(서버, 클라이언트)
```

■ 프로토콜 동작 과정



■ 기능 구현 (30점)

✓ 클라이언트가 데이터를 랜덤한 순서로 전송하지 않을 경우, 0점 처리함.

✓ 실행 결과와 같이 패킷 송수신 과정을 출력하지 않는 경우, 동작하지 않은 것으로 간주하고 채점합니다.

✓ **1. 클라이언트의 파일 읽기, 구조체 배열에 저장 및 화면 출력 (4점)**

- 클라이언트는 주어진 파일을 한 라인 단위로 읽고 순서대로 구조체 배열(DATA file_data[MAX_LINE])에 저장함 (seq번호는 1번부터 시작)
- 구조체 배열의 전체 내용을 화면에 출력
  - 예시) seq [1] First, Hello, this is a test message from the client.

✓ **2. 클라이언트의 FILE_SEND 송신 및 FILE_RES 수신 기능 구현 (4점)**

- 클라이언트는 서버에 접속 후, 랜덤 값(0~9 사이)을 생성하고 구조체 배열에서 해당 랜덤값에 해당하는 데이터를 REQ_PACKET 구조체 변수에 저장 후 서버로 전송함 (1초 간격으로 전송)
- 서버로부터 FILE_RES 메시지를 수신한 경우에도 동일하게 랜덤 값 생성 후 FILE_SEND 메시지를 서버로 전송 (1초 간격)
- 클라이언트의 송수신 내용은 아래와 같이 화면에 출력
  - 예1) [Tx] cmd: 1, SEQ: 9 --> [Rx] cmd: 2, SEQ: 9, result: NOT_OK
  - 예2) [Tx] cmd: 1, SEQ: 1 --> [Rx] cmd: 2, SEQ: 1, result: OK

✓ **3. 서버의 FILE_SEND (정상 seq 번호) 수신 및 응답 기능 구현(18점)**

- 서버는 FILE_SEND 메시지에 포함된 seq 번호가 순서대로 수신되기를 기대함 (seq 1 -> seq 2 -> ... -> seq10 순서)
- 1) 수신된 seq 번호가 기대했던 번호인 경우 (맨 처음 1번 수신)
  - 수신 데이터의 내용을 구조체 배열(DATA file_data[MAX_LINE])의 해당 seq 인덱스에 저장 및 Success Count 값을 화면에 출력 (6점)
  - 예) [Rx] cmd: 1, seq: 1 --> Success Count: 1
  - 다음 번 수신될 seq 번호를 검사하기 위한 별도의 변수 사용

- 2) FILE_RES 메시지 송신 (2점)
  - RES_PACKET의 cmd= FILE_RES, seq = 수신된 seq, result=OK 값을 저장하여 클라이언트로 전송

- 3) 정상적으로 수신된 seq 번호의 개수가 MAX_LINE 개수와 같으면, 기존에 수신된 모든 데이터(DATA file_datat[])를 화면에 출력: 출력 내용 참고 (6점)
  - 출력 내용에는 불필요한 값이 없어야 하며, seq 순서대로 출력되어야 함

```
-----------------------------------------------------------
seq [ 1] First, Hello, this is a test message from the client.
seq [ 2] Second, the system status is currently normal.
seq [ 3] Third, requesting the latest update from the server.
seq [ 4] Fourth, data transmission has started successfully.
seq [ 5] Fifth, please acknowledge receipt of this packet.
```

```
seq [ 6] Sixth, the connection will be closed in five seconds.
seq [ 7] Seventh, sending the collected sensor data now.
seq [ 8] Eighth, error detected during the previous transfer.
seq [ 9] Ninth, ping request sent to verify connectivity.
seq [10] Tenth, Bye!
------------------------------------------------------------
```

- FILE_SEND, FILE_RES 기능 구현이나 화면 출력이 없으면, 해당 점수 없음

- 4) 총 MAX_LINE 개수만큼 데이터를 수신한 경우, **RX_DONE** 메시지를 클라이언트로 전송 (4점)
  - RES_PACKET의 cmd= RX_DONE, seq = 수신된 seq, result=OK 값을 저장하여 클라이언트로 전송 후 아래 메시지 화면에 출력
  - 예) [Tx] RX_DONE

✓ **4. 서버의 비정상 seq 번호(FILE_SEND) 수신 및 응답 기능 구현**(2점)

- 수신된 seq 번호가 기대한 순서와 다른 경우, 아래와 같이 화면에 출력 후 RES_PACKET를 클라이언트로 전송함
  - cmd = FILE_RES, seq는 수신한 seq 번호 그대로 사용, result = NOT_OK
  - 예) [Rx] cmd: 1, seq: 9 --> [Tx] cmd: 2, seq: 9, result: NOT_OK

✓ **5. RX_DONE 및 RX_DONE_ACK 기능 구현** (2점)

- 클라이언트가 RX_DONE 메시지를 수신하면, 서버로 RX_DONE_ACK 메시지로 응답 후 클라이언트 종료
  - [Rx] RX_DONE
  - [Tx] RX_DONE_ACK
- 서버는 RX_DONE_ACK를 수신하면 이를 화면에 출력한 다음 프로그램을 종료
  - 예: [Rx] RX_DONE_ACK

■ 실행 결과 (제공된 동영상 파일 참조)

Server

```
$ ./server 9190
[Rx] cmd: 1, seq:  9 --> [Tx] cmd: 2, seq:  9, result: NOT_OK
[Rx] cmd: 1, seq:  3 --> [Tx] cmd: 2, seq:  3, result: NOT_OK
[Rx] cmd: 1, seq:  9 --> [Tx] cmd: 2, seq:  9, result: NOT_OK
[Rx] cmd: 1, seq:  2 --> [Tx] cmd: 2, seq:  2, result: NOT_OK
[Rx] cmd: 1, seq:  2 --> [Tx] cmd: 2, seq:  2, result: NOT_OK
[Rx] cmd: 1, seq:  7 --> [Tx] cmd: 2, seq:  7, result: NOT_OK

. . . 중간 생략

[Rx] cmd: 1, seq:  1 --> Success Count: 1 --> [Tx] cmd: 2, seq:  1, result: OK
[Rx] cmd: 1, seq:  9 --> [Tx] cmd: 2, seq:  9, result: NOT_OK
[Rx] cmd: 1, seq:  5 --> [Tx] cmd: 2, seq:  5, result: NOT_OK
[Rx] cmd: 1, seq:  7 --> [Tx] cmd: 2, seq:  7, result: NOT_OK
[Rx] cmd: 1, seq:  2 --> Success Count: 2 --> [Tx] cmd: 2, seq:  2, result: OK
[Rx] cmd: 1, seq:  8 --> [Tx] cmd: 2, seq:  8, result: NOT_OK

. . . 중간 생략

[Rx] cmd: 1, seq:  9 --> Success Count: 9 --> [Tx] cmd: 2, seq:  9, result: OK
[Rx] cmd: 1, seq:  5 --> [Tx] cmd: 2, seq:  5, result: NOT_OK
[Rx] cmd: 1, seq:  6 --> [Tx] cmd: 2, seq:  6, result: NOT_OK
[Rx] cmd: 1, seq:  5 --> [Tx] cmd: 2, seq:  5, result: NOT_OK
[Rx] cmd: 1, seq:  5 --> [Tx] cmd: 2, seq:  5, result: NOT_OK
[Rx] cmd: 1, seq: 10 --> Success Count: 10 --> Reassembled All Data
------------------------------------------------------------
seq [ 1] First, Hello, this is a test message from the client.
seq [ 2] Second, the system status is currently normal.
seq [ 3] Third, requesting the latest update from the server.
seq [ 4] Fourth, data transmission has started successfully.
seq [ 5] Fifth, please acknowledge receipt of this packet.
seq [ 6] Sixth, the connection will be closed in five seconds.
seq [ 7] Seventh, sending the collected sensor data now.
seq [ 8] Eighth, error detected during the previous transfer.
seq [ 9] Ninth, ping request sent to verify connectivity.
seq [10] Tenth, Bye!
------------------------------------------------------------
[Tx] RX_DONE --> [Rx] RX_DONE_ACK
Exit Server
$
```

Client

```
$ ./client 127.0.0.1 9190
Read File Data
```

```
------------------------------------------------------------
seq [ 1] First, Hello, this is a test message from the client.
seq [ 2] Second, the system status is currently normal.
seq [ 3] Third, requesting the latest update from the server.
seq [ 4] Fourth, data transmission has started successfully.
seq [ 5] Fifth, please acknowledge receipt of this packet.
seq [ 6] Sixth, the connection will be closed in five seconds.
seq [ 7] Seventh, sending the collected sensor data now.
seq [ 8] Eighth, error detected during the previous transfer.
seq [ 9] Ninth, ping request sent to verify connectivity.
seq [10] Tenth, Bye!
------------------------------------------------------------
[Tx] cmd: 1, SEQ:  9 --> [Rx] cmd: 2, SEQ:  9, result: NOT_OK
[Tx] cmd: 1, SEQ:  3 --> [Rx] cmd: 2, SEQ:  3, result: NOT_OK

. . .  중간 생략 (동영상 참조)

[Tx] cmd: 1, SEQ:  9 --> [Rx] cmd: 2, SEQ:  9, result: OK
[Tx] cmd: 1, SEQ:  5 --> [Rx] cmd: 2, SEQ:  5, result: NOT_OK
[Tx] cmd: 1, SEQ:  6 --> [Rx] cmd: 2, SEQ:  6, result: NOT_OK
[Tx] cmd: 1, SEQ:  5 --> [Rx] cmd: 2, SEQ:  5, result: NOT_OK
[Tx] cmd: 1, SEQ:  5 --> [Rx] cmd: 2, SEQ:  5, result: NOT_OK
[Tx] cmd: 1, SEQ: 10 --> [Rx] RX_DONE
[Tx] RX_DONE_ACK
Exit Client
$
```